

Hirkanet Logging and Elastic Stack Guide

Corrected application logging, Docker collection, Filebeat, Logstash, Elasticsearch, Kibana, retention, security, and operating instructions.

Area	Implementation
Application	Structured JSON, request IDs, redaction, authentication/audit/domain/error events
Collection	Docker json-file rotation and Filebeat 9 filestream container parser
Processing	Logstash JSON parsing, TLS output, malformed-event separation
Storage	TLS-enabled Elasticsearch and 30-day ILM policy
Visualization	Kibana data view, queries, dashboards, and alerts

1. Architecture and event flow

The application and Gunicorn write one JSON object per line to stdout. Docker captures and rotates those files. Filebeat reads the Docker container log format, enriches events with Docker metadata, and drops containers outside the approved allowlist. Logstash parses application JSON and forwards valid records through TLS to Elasticsearch. Invalid JSON is isolated in a dead-letter index. Kibana searches and visualizes the resulting indices.

2. File map

logging_config.py - central formatter, redaction, request correlation, and duration logging.

api/app.py - initialization, login/logout events, readiness checks, and global exception handling.

gunicorn_logging.py - structured access events.

client/routes.py - policy evaluation and batch evaluation telemetry.

admin/routes.py - user-management audit events and secure passwords.

collectors/fortigate_collector.py - collector operational events.

elk/filebeat/filebeat.yml - Filebeat 9 filestream input and filtering.

elk/logstash/pipeline/logstash.conf - parsing, dead-letter routing, ILM, and TLS.

docker-compose.yml - certificates, credentials, port bindings, health checks, and log rotation.

elk/setup-ilm.sh - lifecycle policy installation.

3. Application-side implementation

All modules obtain a logger by module name. Handlers are configured once in **logging_config.py**. The JSON formatter emits UTC timestamps, log level, logger name, service identity, event dataset, request context, approved extra fields, and structured exception information.

Central logger configuration - logging_config.py

```
from logging_config import configure_logging, install_request_logging
configure_logging()
app = Flask(__name__)
install_request_logging(app)
```

Correct business event

```
logger.info(
    "Policy evaluation completed",
    extra={
        "event_type": "policy_evaluation_completed",
        "user_id": str(current_user.id),
        "result_count": len(results),
        "event_duration": duration_ns,
    },
)
```

Exception event

```
try:
    execute_operation()
except EngineError:
    logger.exception(
        "Policy evaluation failed",
        extra={"event_type": "policy_evaluation_failure"},
    )
    raise
```

Level policy

DEBUG: detailed diagnostic state, disabled in normal production. **INFO**: expected lifecycle and business events.

WARNING: recoverable anomaly or suspicious behavior. **ERROR**: failed operation without automatic traceback.

EXCEPTION: failed operation inside an exception handler, including traceback. **CRITICAL**: process or core service cannot continue.

Data handling rules

Do not log passwords, password hashes, salts, API tokens, authorization headers, cookies, CSRF values, database URLs containing credentials, complete sensitive request payloads, or raw FortiGate tokens. The

formatter redacts common secret field names, but callers must still avoid placing confidential values in message strings.

4. Request correlation and access logs

Every Flask request receives an **X-Request-ID**. The ID is returned in the response and added to application events. The request middleware records duration in nanoseconds. Gunicorn emits a separate ECS-oriented access dataset with method, path, status, response bytes, source IP, user agent, and duration.

```
@app.before_request
def start_request():
    g.request_id = request.headers.get("X-Request-ID") or str(uuid.uuid4())
    g.request_started = time.perf_counter()

@app.after_request
def finish_request(response):
    response.headers["X-Request-ID"] = g.request_id
    logger.info("HTTP request completed", extra={
        "event_type": "access",
        "event_duration": duration_ns,
        "http_status_code": response.status_code,
    })
    return response
```

Authentication and audit events

The corrected application records successful and failed authentication, logout, bootstrap administrator creation, user creation, and user deletion. Administrative events include actor and target identifiers without password material. Legacy salted SHA-256 passwords are transparently verified once and migrated to Werkzeug script hashes.

5. Filebeat

The old Filebeat **container** input was replaced because it is not appropriate for Filebeat 9. The project now uses **filestream** with the Docker container parser. Docker metadata is added, and only the application and PostgreSQL containers are retained.

```
filebeat.inputs:
- type: filestream
  id: hirkanet-docker-logs
  paths: ["/var/lib/docker/containers/**/*.log"]
  parsers:
  - container:
    stream: all
    format: docker
processors:
- add_docker_metadata:
  host: unix:///var/run/docker.sock
- drop_event:
  when:
  not:
  or:
  - equals: {container.name: fortigate-app}
  - equals: {container.name: fortigate-db}
output.logstash:
  hosts: ["logstash:5044"]
```

6. Logstash

Logstash parses only application-container events as JSON. Parsed fields are promoted into the event. PostgreSQL records receive a separate service and dataset. JSON failures are tagged and routed to **hirkanet-dead-letter-***. Valid events are written through HTTPS and use the **hirkanet-logs-policy** lifecycle policy.

```
json {
  source => "message"
  target => "[@metadata][parsed]"
  tag_on_failure => ["hirkanet_json_parse_failure"]
}

if "hirkanet_json_parse_failure" in [tags] {
  elasticsearch { index => "hirkanet-dead-letter-%{+YYYY.MM.dd}" }
} else {
  elasticsearch {
    ilm_enabled => true
    ilm_rollover_alias => "hirkanet-logs"
    ilm_policy => "hirkanet-logs-policy"
  }
}
```

7. Elasticsearch and Kibana security

Elasticsearch security and HTTPS are enabled. The setup service creates a certificate authority and an Elasticsearch certificate, then configures the Kibana system account. Elasticsearch and Kibana ports bind to 127.0.0.1. Logstash port 5044 is exposed only to the Compose network. Secrets are read from .env and are not hard-coded in Compose.

Required environment values

```
POSTGRES_PASSWORD=<long random value>
SECRET_KEY=<at least 32 random bytes>
BOOTSTRAP_ADMIN_USERNAME=admin
BOOTSTRAP_ADMIN_PASSWORD=<12+ character strong value>
ELASTIC_PASSWORD=<long random value>
KIBANA_PASSWORD=<different long random value>
```

Important bootstrap rule

After the first administrator is created, remove **BOOTSTRAP_ADMIN_USERNAME** and **BOOTSTRAP_ADMIN_PASSWORD** from the runtime environment. The application no longer creates an admin/admin account.

8. Deployment procedure

1. Copy .env.example to .env and replace every placeholder.
2. On Linux, run: `sudo sysctl -w vm.max_map_count=262144`
3. Start: `docker compose up -d --build`
4. Inspect: `docker compose ps` and `docker compose logs --tail=100 app filebeat logstash elasticsearch kibana`
5. Install the ILM policy using the command in ELK-SETUP.md.
6. Generate application traffic, then create a Kibana data view named hirkanet-logs* using @timestamp.
7. Remove bootstrap administrator environment variables after successful first login.

9. Operational queries

Purpose	KQL
Application errors	<code>service.name: "hirkanet" and log.level: ("error" or "critical")</code>
Authentication failures	<code>event_type: "authentication_failure"</code>
Policy evaluation completion	<code>event_type: "policy_evaluation_completed"</code>
Slow events	<code>event.duration > 1000000000</code>
Single request	<code>request.id: "<request-id>"</code>
Parsing failures	<code>tags: "hirkanet_json_parse_failure"</code>

10. Dashboards and alerts

Dashboards: request volume, HTTP status distribution, p50/p95/p99 latency, top paths, errors by type, login failures, evaluation count and latency, parse failures, Elasticsearch disk use, and index growth.

Alerts: critical events, repeated HTTP 500 responses, authentication failure spikes, missing application logs, any JSON parsing failures, Filebeat output errors, unhealthy Elasticsearch cluster, disk above 80%, and policy-evaluation latency above the service objective.

11. Verification performed

The corrected repository was checked with Python bytecode compilation, Compose YAML parsing, scans for obsolete Filebeat input and insecure hard-coded defaults, and PDF rendering. Live Docker/Elastic execution and dependency-based pytest execution could not be completed in the artifact environment because no Docker daemon or external Python package index was available. The package includes tests for password verification and JSON redaction for execution in CI or a development environment with dependencies installed.

12. Recommended CI commands

```
python -m pip install -r requirements.txt
python -m compileall -q .
python -m pytest -q
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=200 app filebeat logstash elasticsearch kibana
```

13. Final implementation notes

The application remains backward compatible with existing legacy password records through transparent migration. Database table creation remains available for first-run compatibility; for production schema evolution, introduce Alembic or Flask-Migrate before making future model changes. TLS is enabled between Logstash and Elasticsearch; Filebeat-to-Logstash remains inside the isolated Compose network without TLS. Add Beats TLS when Filebeat and Logstash are separated across hosts or untrusted networks.